**CSE47201 Computer Vision Assignment 1 [150 points]**

**Deadline : until Nov. 16, 23:59**

- If you have any questions, please post it on the blackboard/discussion/ 'assignment 1' board.

1. [30 points] Please download the training/testing database using the link below, respectively:
   https://drive.google.com/file/d/1FmnFOJBN5ihcKC_WW7T99bN-reTrVu0-/view?usp=sharing

   Please use the training database when training your model while using the testing database when evaluating your model. Using the ``Bag-of-words'' model, please implement an object classifier that can classify 20 objects defined in the downloaded dataset.

   Accuracy measurement is defined as follows:

   $$Accuracy = \frac{(\#\ of\ samples\ which\ are\ correctly\ predicted)}{(\#\ of\ total\ samples)}$$

   Please calculate the 'accuracy' of your 20 object classifier using the 'SVM' on the downloaded testing data. Please use the multi-class SVM by calling the function below (This SVM uses the RBF kernel by default.):

   ```
   svm = cv2.ml.SVM_create()
   svm.train(hists, cv2.ml.ROW_SAMPLE, np.array(train_labels))
   svm.save(svm_model_file)
   ```

   Also, please make sure that the L1 normalization is applied on the obtained histogram (BOW) representation before applying SVM.

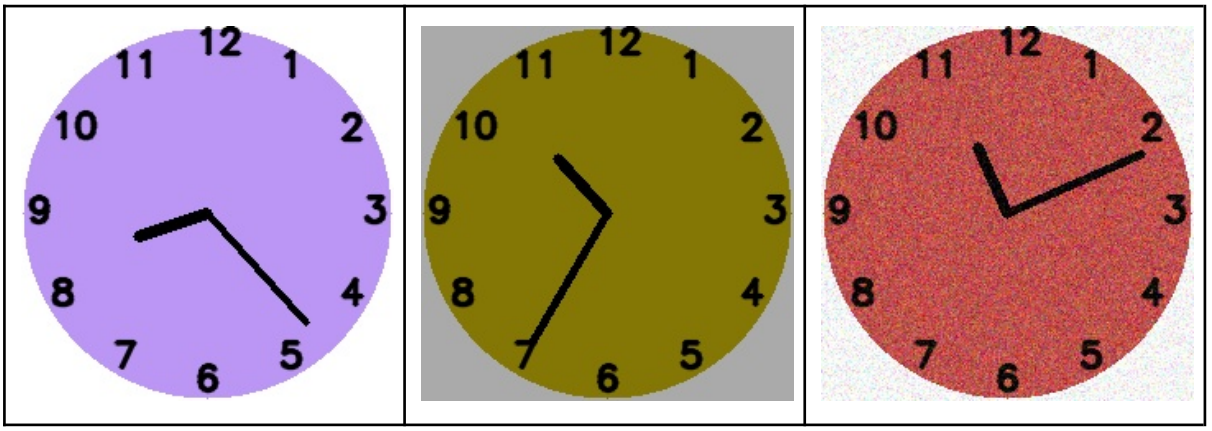
   ```
   hist = np.float32(hist) / np.float32(np.sum(hist))
   ```

2. [15 points] Please find the relationship between the 'number of codewords' and the overall 'accuracy' of your method developed in Problem 1. You need to draw 2D graphs by changing # of codewords and calculating corresponding accuracy. Please try more than 3 different # of codewords.

3. [5 points] Please use a 'histogram intersection kernel' for the 'SVM' in problem 1 using the 'setKernel' command below and perform the same thing as problem 1 to measure their accuracy.

   ```
   svm.setKernel(cv2.ml.SVM_INTER)
   ```

4. [30 points] Using the OpenCV library, please write a code for drawing clock images given two arguments (ie. hour, minute) and meeting the conditions, as follows:



[Conditions]

- The image size is 227 pixels x 227 pixels x 3 channels.
- The clock is a circle and it has a radius of 112 pixels.
- The clock and background have random colors (but they are always different) but it would not be black.
- The clock and background images can have random noises as the 3rd figure.
- The minute and hour hands start from the center of the image.
- The black-colored hour hand length is 45 pixels and the thickness is 5.
- The black-colored minute hand length is 90 pixels and the thickness is 3.
- Digits of 1~12 in black color are located as the clock and having fonts as follows:

```
font = cv2.FONT_HERSHEY_SIMPLEX
font_scale = 0.7
font_thickness = 2
cv2.putText(.. , .., .., font, font_scale, (0, 0, 0), font_thickness,
cv2.LINE_AA)
```

Example image are uploaded here:
https://drive.google.com/drive/folders/1_zdugouxC2hSalLsv_iCd5pbKHSShhr1?usp=sharing

5. [20 points] Please write a code that implements the CNN network predicting the hour and the minute from the 227x227x3 sized images that we obtained from Problem 4.

   - To solve the problem, you need to propose the CNN architecture and train and test it. You can train the network using either classification loss or regression loss. Or you can also use both, if necessary by adding two losses.
   - You are allowed to implement the code to load and manipulate the already pre-trained network (e.g. ImageNet pre-trained ResNet).

[Tips] You can decide the output of the CNN network not being the raw hour and minute values. Rather, you can formulate it to output the degrees of hour and minute hands. Or, you can estimate their x and y coordinate values, etc.
[Tips] In a regression task, the raw output space could be too wide when you try to estimate values from -inf to inf. It can hinder network training. To relieve the issue, one can normalize the ground-truth/output values to be confined in 0 and 1 ranges. Towards this, one can use the sigmoid layer as the last layer of the proposed network and make a formula to normalize ground-truths. Then, one can train the proposed network using the MSE loss by closing the distance between normalized ground-truth and normalized predicted values. During inference, one can develop and apply the un-normalization formula that reconstructs the trained network's normalized output values to be in the original scale.

6. [10 points] Please implement a code for training the CNN network using the data you created in Problem 4 and train it. You can generate a sufficient amount of training data to properly train your network. You need to save the learned weights of CNN network and submit it in the submission file in the form of .pth file.

7. [10 points] Please implement a code for testing the CNN network for an image. The code inputs an image and it has to output the proper hour and minute.

8. [15 points] Please analyze the successful and failure cases of your CNN network. How accurately can it detect the hour and the minute from the images? In our preliminary implementation, we noticed that deep networks could be highly accurate in inferring hours and minutes. We will measure the accuracy of your networks using 100 random samples, allowing 5 minutes as the error margin: for example we will regard it as correct if you estimate 10h 40m for images having 10h 38m. The accuracy of the model will count for 15 points. If the accuracy is too low, some points could be also deducted in problems 5-6, as it implies that the network design is non-effective.

9. [15 points] Please note and apply the below details.
    a. Send all files(.zip) via **Blackboard**.
    b. In **20251111_SeungryulBaek_cv_ass1.zip** file, you need to have python source code (*.py), a pre-trained network weight (*.pth) and necessary binaries (*.npy, *.xml files) contained in the zip file.
    c. You need to submit a report (report.pdf or report.docx) within 10 pages, properly explaining your solutions and analyzing results for each problem.
    d. Code should have some **comments** to increase the readability.
    e. Your pre-trained weights in .pth need to be properly loaded.